

Lab1

Создано системой Doxygen 1.17.0

1	Дерево директорий	1
1.1	Алфавитный указатель директорий	1
2	Алфавитный указатель классов	3
2.1	Классы	3
3	Список файлов	5
3.1	Файлы	5
4	Директории	7
4.1	Содержание директории DataSetMaker	7
4.2	Содержание директории Sorts	7
5	Классы	9
5.1	Структура Товар	9
5.1.1	Подробное описание	9
5.1.2	Данные класса	9
5.1.2.1	country	9
5.1.2.2	money	10
5.1.2.3	name	10
5.1.2.4	volume	10
6	Файлы	11
6.1	Файл DataSetMaker/dsmaker.cpp	11
6.1.1	Функции	11
6.1.1.1	main()	11
6.2	dsmaker.cpp	12
6.3	Файл Sorts/bubble.cpp	12
6.3.1	Функции	13
6.3.1.1	bubbleSort()	13
6.4	bubble.cpp	13
6.5	Файл Sorts/main.cpp	13
6.5.1	Функции	14
6.5.1.1	copyArray()	14
6.5.1.2	main()	14
6.5.1.3	readData()	16
6.5.1.4	writeData()	17
6.6	main.cpp	17
6.7	Файл Sorts/merge.cpp	19
6.7.1	Функции	19
6.7.1.1	merge()	19
6.7.1.2	merge_sort()	20
6.8	merge.cpp	21
6.9	Файл Sorts/MySortings.h	21
6.9.1	Функции	22

6.9.1.1 bubbleSort()	22
6.9.1.2 merge()	22
6.9.1.3 merge_sort()	24
6.9.1.4 shakerSort()	24
6.10 MySortings.h	25
6.11 Файл Sorts/oper.cpp	27
6.11.1 Функции	27
6.11.1.1 operator<()	27
6.11.1.2 operator<=()	28
6.11.1.3 operator>()	28
6.11.1.4 operator>=()	29
6.12 oper.cpp	29
6.13 Файл Sorts/oper.h	29
6.13.1 Функции	30
6.13.1.1 operator<()	30
6.13.1.2 operator<=()	31
6.13.1.3 operator>()	31
6.13.1.4 operator>=()	32
6.14 oper.h	32
6.15 Файл Sorts/shaker.cpp	32
6.15.1 Функции	33
6.15.1.1 shakerSort()	33
6.16 shaker.cpp	33
Предметный указатель	35

Глава 1

Дерево директорий

1.1 Алфавитный указатель директорий

DataSetMaker	7
dsmaker.cpp	11
Sorts	7
bubble.cpp	12
main.cpp	13
merge.cpp	19
MySortings.h	21
oper.cpp	27
oper.h	29
shaker.cpp	32

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

Tovar	
структура товара для экспорта	9

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

DataSetMaker/ dsmaker.cpp	11
Sorts/ bubble.cpp	12
Sorts/ main.cpp	13
Sorts/ merge.cpp	19
Sorts/ MySortings.h	21
Sorts/ oper.cpp	27
Sorts/ oper.h	29
Sorts/ shaker.cpp	32

Глава 4

Директории

4.1 Содержание директории DataSetMaker

Файлы

- файл [dsmaker.cpp](#)

4.2 Содержание директории Sorts

Файлы

- файл [bubble.cpp](#)
- файл [main.cpp](#)
- файл [merge.cpp](#)
- файл [MySortings.h](#)
- файл [oper.cpp](#)
- файл [oper.h](#)
- файл [shaker.cpp](#)

Глава 5

Классы

5.1 Структура Tovar

структура товара для экспорта

```
#include <oper.h>
```

Открытые атрибуты

- string [name](#)
наименование товара
- string [country](#)
страна экспорта
- int [volume](#)
объем поставляемой продукции
- int [money](#)
сумма поставки в рублях

5.1.1 Подробное описание

структура товара для экспорта

См. определение в файле [oper.h](#) строка [9](#)

5.1.2 Данные класса

5.1.2.1 country

```
string Tovar::country
```

страна экспорта

См. определение в файле [oper.h](#) строка [12](#)

Используется в [operator<\(\)](#), [readData\(\)](#) и [writeData\(\)](#).

5.1.2.2 money

`int Товар::money`

сумма поставки в рублях

См. определение в файле [oper.h](#) строка 14

Используется в [readData\(\)](#) и [writeData\(\)](#).

5.1.2.3 name

`string Товар::name`

наименование товара

См. определение в файле [oper.h](#) строка 11

Используется в [operator<\(\)](#), [readData\(\)](#) и [writeData\(\)](#).

5.1.2.4 volume

`int Товар::volume`

объем поставляемой продукции

См. определение в файле [oper.h](#) строка 13

Используется в [operator<\(\)](#), [readData\(\)](#) и [writeData\(\)](#).

Объявления и описания членов структуры находятся в файле:

- [Sorts/oper.h](#)

Глава 6

Файлы

6.1 Файл DataSetMaker/dsmaker.cpp

```
#include <iostream>
#include <string>
#include <cstdlib>
#include <fstream>
```

Граф включаемых заголовочных файлов для dsmaker.cpp:

Функции

- `int main ()`

6.1.1 Функции

6.1.1.1 main()

`int main ()`

См. определение в файле [dsmaker.cpp](#) строка 8

```
00009 {
00010     ofstream fout("ds.txt");
00011
00012     string tovar[25] =
00013     {
00014         "Oil", "Gas", "Coal", "Steel", "Aluminum",
00015         "Nickel", "Copper", "Wheat", "Barley", "Corn",
00016         "Fertilizer", "Timber", "Wood", "Paper", "Diesel",
00017         "Petrol", "Gold", "Silver", "Palladium", "Platinum",
00018         "Fish", "Seafood", "Machinery", "Chemicals", "Rubber"
00019     };
00020
00021     string cs[25] =
00022     {
00023         "China", "India", "Turkey", "Kazakhstan", "Belarus",
00024         "Uzbekistan", "Armenia", "Kyrgyzstan", "Egypt", "UAE",
00025         "SaudiArabia", "Iran", "Pakistan", "Brazil", "Vietnam",
00026         "Indonesia", "Mongolia", "Serbia", "Algeria", "SouthAfrica",
00027         "Thailand", "Malaysia", "Bangladesh", "Mexico", "Morocco"
00028     };
00029
00030     for (int i = 0; i < 100000; i++)
00031     {
00032         string product = tovar[rand() % 25];
```

```

00033     string country = cs[rand() % 25];
00034     int volume = (rand() % 10000) + 1;
00035     int money = (rand() % 5000000) + 10000;
00036
00037     fout << product << ";" << country << ";" << volume << ";" << money << endl;
00038 }
00039
00040 fout.close();
00041
00042 return 0;
00043 }

```

6.2 dsmaker.cpp

[См. документацию.](#)

```

00001 #include <iostream>
00002 #include <string>
00003 #include <cstdlib>
00004 #include <fstream>
00005
00006 using namespace std;
00007
00008 int main()
00009 {
00010     ofstream fout("ds.txt");
00011
00012     string tovar[25] =
00013     {
00014         "Oil", "Gas", "Coal", "Steel", "Aluminum",
00015         "Nickel", "Copper", "Wheat", "Barley", "Corn",
00016         "Fertilizer", "Timber", "Wood", "Paper", "Diesel",
00017         "Petrol", "Gold", "Silver", "Palladium", "Platinum",
00018         "Fish", "Seafood", "Machinery", "Chemicals", "Rubber"
00019     };
00020
00021     string cs[25] =
00022     {
00023         "China", "India", "Turkey", "Kazakhstan", "Belarus",
00024         "Uzbekistan", "Armenia", "Kyrgyzstan", "Egypt", "UAE",
00025         "SaudiArabia", "Iran", "Pakistan", "Brazil", "Vietnam",
00026         "Indonesia", "Mongolia", "Serbia", "Algeria", "SouthAfrica",
00027         "Thailand", "Malaysia", "Bangladesh", "Mexico", "Morocco"
00028     };
00029
00030     for (int i = 0; i < 100000; i++)
00031     {
00032         string product = tovar[rand() % 25];
00033         string country = cs[rand() % 25];
00034         int volume = (rand() % 10000) + 1;
00035         int money = (rand() % 5000000) + 10000;
00036
00037         fout << product << ";" << country << ";" << volume << ";" << money << endl;
00038     }
00039
00040     fout.close();
00041
00042     return 0;
00043 }

```

6.3 Файл Sorts/bubble.cpp

Функции

- `template<class T>`
void `bubbleSort` (T a[], long size)

6.3.1 Функции

6.3.1.1 bubbleSort()

```
template<class T>
void bubbleSort (
    T a[],
    long size)
```

См. определение в файле [bubble.cpp](#) строка 4

```
00005 {
00006     long i, j;
00007     T x;
00008     for (i = 0; i < size; i++)
00009     {
00010         for (j = size - 1; j > i; j--)
00011         {
00012             if (a[j - 1] > a[j])
00013             {
00014                 x = a[j - 1];
00015                 a[j - 1] = a[j];
00016                 a[j] = x;
00017             }
00018         }
00019     }
00020 }
```

Используется в [main\(\)](#).

Граф вызова функции:

6.4 bubble.cpp

[См. документацию.](#)

```
00001
00002
00003 template<class T>
00004 void bubbleSort(T a[], long size)
00005 {
00006     long i, j;
00007     T x;
00008     for (i = 0; i < size; i++)
00009     {
00010         for (j = size - 1; j > i; j--)
00011         {
00012             if (a[j - 1] > a[j])
00013             {
00014                 x = a[j - 1];
00015                 a[j - 1] = a[j];
00016                 a[j] = x;
00017             }
00018         }
00019     }
00020 }
```

6.5 Файл Sorts/main.cpp

```
#include <fstream>
#include <string>
#include <chrono>
#include <algorithm>
#include "oper.h"
#include "MySortings.h"
```

Граф включаемых заголовочных файлов для main.cpp:

Функции

- `int readData (Tovar a[], int needSize)`
считывает товары из файла ds.txt
- `void copyArray (Tovar from[], Tovar to[], int size)`
копирует один массив товаров в другой
- `void writeData (Tovar a[], int size)`
записывает отсортированный массив товаров в файл sorted.txt
- `int main ()`
главная функция программы

6.5.1 Функции

6.5.1.1 copyArray()

```
void copyArray (
    Tovar from[],
    Tovar to[],
    int size)
```

копирует один массив товаров в другой

Аргументы

from	исходный массив
to	массив-приемник
size	размер массива

См. определение в файле [main.cpp](#) строка 57

```
00058 {
00059     for (int i = 0; i < size; i++)
00060     {
00061         //копируем элемент массива
00062         to[i] = from[i];
00063     }
00064 }
```

Используется в [main\(\)](#).

Граф вызова функции:

6.5.1.2 main()

```
int main ()
```

главная функция программы

Возвращает

код завершения программы

См. определение в файле [main.cpp](#) строка 91

```

00092 {
00093     const int maxSize = 100000;
00094
00095     int sizes[12] =
00096     {
00097         100, 500, 1000, 2500, 5000, 10000,
00098         20000, 40000, 60000, 80000, 90000, 100000
00099     };
00100
00101     //создаем массив для исходных данных
00102     Tovar* original = new Tovar[maxSize];
00103
00104     //создаем массив для сортировки
00105     Tovar* a = new Tovar[maxSize];
00106
00107     ofstream result("result.txt", ios::trunc);
00108
00109     result << "size bubble shaker merge std_sort" << endl;
00110
00111     for (int i = 0; i < 12; i++)
00112     {
00113         //считываем нужное количество записей
00114         int size = readData(original, sizes[i]);
00115
00116         //копируем исходный массив перед сортировкой
00117         copyArray(original, a, size);
00118
00119         auto start = high_resolution_clock::now();
00120         bubbleSort(a, size);
00121         auto end = high_resolution_clock::now();
00122
00123         long long bubbleTime = duration_cast<microseconds>(end - start).count();
00124
00125         //копируем исходный массив перед сортировкой
00126         copyArray(original, a, size);
00127
00128         start = high_resolution_clock::now();
00129         shakerSort(a, size);
00130         end = high_resolution_clock::now();
00131
00132         long long shakerTime = duration_cast<microseconds>(end - start).count();
00133
00134         //копируем исходный массив перед сортировкой
00135         copyArray(original, a, size);
00136
00137         start = high_resolution_clock::now();
00138         merge_sort(a, 0, size - 1);
00139         end = high_resolution_clock::now();
00140
00141         long long mergeTime = duration_cast<microseconds>(end - start).count();
00142
00143         //копируем исходный массив перед сортировкой
00144         copyArray(original, a, size);
00145
00146         start = high_resolution_clock::now();
00147         sort(a, a + size);
00148         end = high_resolution_clock::now();
00149
00150         long long stdSortTime = duration_cast<microseconds>(end - start).count();
00151
00152         //записываем результаты замеров
00153         result << size << " "
00154             << bubbleTime << " "
00155             << shakerTime << " "
00156             << mergeTime << " "
00157             << stdSortTime << endl;
00158     }
00159
00160     //считываем весь набор данных
00161     int size = readData(original, maxSize);
00162
00163     //сортируем итоговый массив
00164     merge_sort(original, 0, size - 1);
00165
00166     //записываем отсортированные данные
00167     writeData(original, size);
00168
00169     result.close();
00170
00171     //освобождаем память

```

```

00172     delete[] original;
00173
00174     //освобождаем память
00175     delete[] a;
00176
00177     return 0;
00178 }

```

Перекрестные ссылки `bubbleSort()`, `copyArray()`, `merge_sort()`, `readData()`, `shakerSort()` и `writeData()`.

Граф вызовов:

6.5.1.3 readData()

```

int readData (
    Tovar a[],
    int needSize)

```

считывает товары из файла ds.txt

Аргументы

a	массив товаров
needSize	нужное количество записей

Возвращает

количество считанных записей

См. определение в файле `main.cpp` строка 18

```

00019 {
00020     ifstream fin("../DataSetMaker/ds.txt");
00021
00022     string line;
00023     int i = 0;
00024
00025     while (getline(fin, line) && i < needSize)
00026     {
00027         int p1 = line.find(';');
00028         int p2 = line.find(':', p1 + 1);
00029         int p3 = line.find(':', p2 + 1);
00030
00031         //считываем название товара
00032         a[i].name = line.substr(0, p1);
00033
00034         //считываем страну
00035         a[i].country = line.substr(p1 + 1, p2 - p1 - 1);
00036
00037         //считываем объем продукции
00038         a[i].volume = stoi(line.substr(p2 + 1, p3 - p2 - 1));
00039
00040         //считываем сумму в рублях
00041         a[i].money = stoi(line.substr(p3 + 1));
00042
00043         i++;
00044     }
00045
00046     fin.close();
00047
00048     return i;
00049 }

```

Перекрестные ссылки `Tovar::country`, `Tovar::money`, `Tovar::name` и `Tovar::volume`.

Используется в `main()`.

Граф вызова функции:

6.5.1.4 writeData()

```
void writeData (
    Tovar a[],
    int size)
```

записывает отсортированный массив товаров в файл sorted.txt

Аргументы

a	массив товаров
size	размер массива

См. определение в файле [main.cpp](#) строка 71

```
00072 {
00073     ofstream fout("sorted.txt", ios::trunc);
00074
00075     for (int i = 0; i < size; i++)
00076     {
00077         //записываем один товар
00078         fout << a[i].name << ",";
00079         << a[i].country << ",";
00080         << a[i].volume << ",";
00081         << a[i].money << endl;
00082     }
00083     fout.close();
00084 }
00085 }
```

Перекрестные ссылки [Tovar::country](#), [Tovar::money](#), [Tovar::name](#) и [Tovar::volume](#).

Используется в [main\(\)](#).

Граф вызова функции:

6.6 main.cpp

[См. документацию.](#)

```
00001 #include <fstream>
00002 #include <string>
00003 #include <chrono>
00004 #include <algorithm>
00005
00006 #include "oper.h"
00007 #include "MySortings.h"
00008
00009 using namespace std;
00010 using namespace chrono;
00011
00018 int readData(Tovar a[], int needSize)
00019 {
00020     ifstream fin("../DataSetMaker/ds.txt");
00021
00022     string line;
00023     int i = 0;
00024
00025     while (getline(fin, line) && i < needSize)
00026     {
00027         int p1 = line.find(';');
00028         int p2 = line.find(':', p1 + 1);
00029         int p3 = line.find(':', p2 + 1);
00030
00031         //считываем название товара
00032         a[i].name = line.substr(0, p1);
00033
00034         //считываем страну
00035         a[i].country = line.substr(p1 + 1, p2 - p1 - 1);
00036     }
```

```

00037     //считываем объем продукции
00038     a[i].volume = stoi(line.substr(p2 + 1, p3 - p2 - 1));
00039
00040     //считываем сумму в рублях
00041     a[i].money = stoi(line.substr(p3 + 1));
00042
00043     i++;
00044 }
00045
00046 fin.close();
00047
00048 return i;
00049 }
00050
00051 void copyArray(Tovar from[], Tovar to[], int size)
00052 {
00053     for (int i = 0; i < size; i++)
00054     {
00055         //копируем элемент массива
00056         to[i] = from[i];
00057     }
00058 }
00059
00060 void writeData(Tovar a[], int size)
00061 {
00062     ofstream fout("sorted.txt", ios::trunc);
00063
00064     for (int i = 0; i < size; i++)
00065     {
00066         //записываем один товар
00067         fout << a[i].name << " ";
00068         << a[i].country << " ";
00069         << a[i].volume << " ";
00070         << a[i].money << endl;
00071     }
00072
00073     fout.close();
00074 }
00075
00076 int main()
00077 {
00078     const int maxSize = 100000;
00079
00080     int sizes[12] =
00081     {
00082         100, 500, 1000, 2500, 5000, 10000,
00083         20000, 40000, 60000, 80000, 90000, 100000
00084     };
00085
00086     //создаем массив для исходных данных
00087     Tovar* original = new Tovar[maxSize];
00088
00089     //создаем массив для сортировки
00090     Tovar* a = new Tovar[maxSize];
00091
00092     ofstream result("result.txt", ios::trunc);
00093
00094     result << "size bubble shaker merge std_sort" << endl;
00095
00096     for (int i = 0; i < 12; i++)
00097     {
00098         //считываем нужное количество записей
00099         int size = readData(original, sizes[i]);
00100
00101         //копируем исходный массив перед сортировкой
00102         copyArray(original, a, size);
00103
00104         auto start = high_resolution_clock::now();
00105         bubbleSort(a, size);
00106         auto end = high_resolution_clock::now();
00107
00108         long long bubbleTime = duration_cast<microseconds>(end - start).count();
00109
00110         //копируем исходный массив перед сортировкой
00111         copyArray(original, a, size);
00112
00113         start = high_resolution_clock::now();
00114         shakerSort(a, size);
00115         end = high_resolution_clock::now();
00116
00117         long long shakerTime = duration_cast<microseconds>(end - start).count();
00118
00119         //копируем исходный массив перед сортировкой
00120         copyArray(original, a, size);
00121
00122         start = high_resolution_clock::now();
00123         merge_sort(a, 0, size - 1);
00124
00125

```

```

00139     end = high_resolution_clock::now();
00140
00141     long long mergeTime = duration_cast<microseconds>(end - start).count();
00142
00143     //копируем исходный массив перед сортировкой
00144     copyArray(original, a, size);
00145
00146     start = high_resolution_clock::now();
00147     sort(a, a + size);
00148     end = high_resolution_clock::now();
00149
00150     long long stdSortTime = duration_cast<microseconds>(end - start).count();
00151
00152     //записываем результаты замеров
00153     result << size << " "
00154           << bubbleTime << " "
00155           << shakerTime << " "
00156           << mergeTime << " "
00157           << stdSortTime << endl;
00158 }
00159
00160 //считываем весь набор данных
00161 int size = readData(original, maxSize);
00162
00163 //сортируем итоговый массив
00164 merge_sort(original, 0, size - 1);
00165
00166 //записываем отсортированные данные
00167 writeData(original, size);
00168
00169 result.close();
00170
00171 //освобождаем память
00172 delete[] original;
00173
00174 //освобождаем память
00175 delete[] a;
00176
00177 return 0;
00178 }

```

6.7 Файл Sorts/merge.cpp

Функции

- `template<class T>`
`void merge (T a[], long low, long mid, long high)`
- `template<class T>`
`void merge_sort (T a[], long low, long high)`

6.7.1 Функции

6.7.1.1 merge()

```

template<class T>
void merge (
    T a[],
    long low,
    long mid,
    long high)

```

См. определение в файле [merge.cpp](#) строка 3

```

00004 {
00005     T* b = new T[high + 1 - low];
00006     long h, i, j, k;
00007     h = low;
00008     i = 0;

```

```

00009   j = mid + 1;
00010
00011   while ((h <= mid) && (j <= high))
00012   {
00013       if (a[h] <= a[j])
00014       {
00015           b[i] = a[h];
00016           h++;
00017       }
00018       else
00019       {
00020           b[i] = a[j];
00021           j++;
00022       }
00023       i++;
00024   }
00025
00026   if (h > mid)
00027   {
00028       for (k = j; k <= high; k++)
00029       {
00030           b[i] = a[k];
00031           i++;
00032       }
00033   }
00034   else
00035   {
00036       for (k = h; k <= mid; k++)
00037       {
00038           b[i] = a[k];
00039           i++;
00040       }
00041   }
00042   // Prints into the original array
00043   for (k = 0; k <= high - low; k++)
00044   {
00045       a[k + low] = b[k];
00046   }
00047   delete[] b;
00048 }

```

Используется в [merge_sort\(\)](#).

Граф вызова функции:

6.7.1.2 merge_sort()

```

template<class T>
void merge_sort (
    T a[],
    long low,
    long high)

```

См. определение в файле [merge.cpp](#) строка 50

```

00051 {
00052     long mid;
00053     if (low < high)
00054     {
00055         mid = (low + high) / 2;
00056         merge_sort(a, low, mid);
00057         merge_sort(a, mid + 1, high);
00058         merge(a, low, mid, high);
00059     }
00060 }

```

Перекрестные ссылки [merge\(\)](#) и [merge_sort\(\)](#).

Используется в [main\(\)](#) и [merge_sort\(\)](#).

Граф вызовов: Граф вызова функции:

6.8 merge.cpp

[См. документацию.](#)

```

00001
00002
00003 template<class T> void merge(T a[], long low, long mid, long high)
00004 {
00005     T* b = new T[high + 1 - low];
00006     long h, i, j, k;
00007     h = low;
00008     i = 0;
00009     j = mid + 1;
00010
00011     while ((h <= mid) && (j <= high))
00012     {
00013         if (a[h] <= a[j])
00014         {
00015             b[i] = a[h];
00016             h++;
00017         }
00018         else
00019         {
00020             b[i] = a[j];
00021             j++;
00022         }
00023         i++;
00024     }
00025
00026     if (h > mid)
00027     {
00028         for (k = j; k <= high; k++)
00029         {
00030             b[i] = a[k];
00031             i++;
00032         }
00033     }
00034     else
00035     {
00036         for (k = h; k <= mid; k++)
00037         {
00038             b[i] = a[k];
00039             i++;
00040         }
00041     }
00042     // Prints into the original array
00043     for (k = 0; k <= high - low; k++)
00044     {
00045         a[k + low] = b[k];
00046     }
00047     delete[] b;
00048 }
00049
00050 template<class T> void merge_sort(T a[], long low, long high)
00051 {
00052     long mid;
00053     if (low < high)
00054     {
00055         mid = (low + high) / 2;
00056         merge_sort(a, low, mid);
00057         merge_sort(a, mid + 1, high);
00058         merge(a, low, mid, high);
00059     }
00060 }

```

6.9 Файл Sorts/MySortings.h

Граф файлов, в которые включается этот файл:

Функции

- `template<class T>`
`void bubbleSort (T a[], int size)`
 сортирует массив методом пузырька

- `template<class T>`
`void shakerSort (T a[], int size)`
 сортирует массив шейкерным методом
- `template<class T>`
`void merge (T a[], int low, int mid, int high)`
 сливает две отсортированные части массива
- `template<class T>`
`void merge_sort (T a[], int low, int high)`
 сортирует массив методом слияния

6.9.1 Функции

6.9.1.1 bubbleSort()

```
template<class T>
void bubbleSort (
    T a[],
    int size)
```

сортирует массив методом пузырька

Параметры шаблона

T	тип элементов массива
---	-----------------------

Аргументы

a	массив для сортировки
size	размер массива

См. определение в файле [MySortings.h](#) строка 10

```
00011 {
00012     T x;
00013
00014     for (int i = 0; i < size; i++)
00015     {
00016         for (int j = size - 1; j > i; j--)
00017         {
00018             //меняем соседние элементы местами
00019             if (a[j - 1] > a[j])
00020             {
00021                 x = a[j - 1];
00022                 a[j - 1] = a[j];
00023                 a[j] = x;
00024             }
00025         }
00026     }
00027 }
```

6.9.1.2 merge()

```
template<class T>
void merge (
    T a[],
    int low,
```

```
int mid,
int high)
```

сливает две отсортированные части массива

Параметры шаблона

T	тип элементов массива
---	-----------------------

Аргументы

a	массив
low	левая граница
mid	середина массива
high	правая граница

См. определение в файле [MySortings.h](#) строка 86

```
00087 {
00088     T* b = new T[high + 1 - low];
00089
00090     int h = low;
00091     int i = 0;
00092     int j = mid + 1;
00093
00094     while ((h <= mid) && (j <= high))
00095     {
00096         //выбираем меньший элемент
00097         if (a[h] <= a[j])
00098         {
00099             b[i] = a[h];
00100             h++;
00101         }
00102         else
00103         {
00104             b[i] = a[j];
00105             j++;
00106         }
00107         i++;
00108     }
00109
00110     if (h > mid)
00111     {
00112         for (int k = j; k <= high; k++)
00113         {
00114             //копируем остаток правой части
00115             b[i] = a[k];
00116             i++;
00117         }
00118     }
00119     else
00120     {
00121         for (int k = h; k <= mid; k++)
00122         {
00123             //копируем остаток левой части
00124             b[i] = a[k];
00125             i++;
00126         }
00127     }
00128
00129     for (int k = 0; k <= high - low; k++)
00130     {
00131         //возвращаем элементы в исходный массив
00132         a[k + low] = b[k];
00133     }
00134
00135     delete[] b;
00137 }
```

Используется в [merge_sort\(\)](#).

Граф вызова функции:

6.9.1.3 merge_sort()

```
template<class T>
void merge_sort (
    T a[],
    int low,
    int high)
```

сортирует массив методом слияния

Параметры шаблона

T	тип элементов массива
---	-----------------------

Аргументы

a	массив для сортировки
low	левая граница сортировки
high	правая граница сортировки

См. определение в файле [MySortings.h](#) строка 147

```
00148 {
00149     if (low < high)
00150     {
00151         //делим массив на две части
00152         int mid = (low + high) / 2;
00153
00154         merge_sort(a, low, mid);
00155         merge_sort(a, mid + 1, high);
00156
00157         //сливаем отсортированные части
00158         merge(a, low, mid, high);
00159     }
00160 }
```

Перекрестные ссылки [merge\(\)](#) и [merge_sort\(\)](#).

Используется в [merge_sort\(\)](#).

Граф вызовов: [Граф вызова функции](#):

6.9.1.4 shakerSort()

```
template<class T>
void shakerSort (
    T a[],
    int size)
```

сортирует массив шейкерным методом

Параметры шаблона

T	тип элементов массива
---	-----------------------

Аргументы

a	массив для сортировки
size	размер массива

См. определение в файле [MySortings.h](#) строка 36

```

00037 {
00038     int j;
00039     int k = size - 1;
00040     int lb = 1;
00041     int ub = size - 1;
00042     T x;
00043
00044     do
00045     {
00046         for (j = ub; j > 0; j--)
00047         {
00048             //проход справа налево
00049             if (a[j - 1] > a[j])
00050             {
00051                 x = a[j - 1];
00052                 a[j - 1] = a[j];
00053                 a[j] = x;
00054                 k = j;
00055             }
00056         }
00057         lb = k + 1;
00059         for (j = 1; j <= ub; j++)
00060         {
00061             //проход слева направо
00062             if (a[j - 1] > a[j])
00063             {
00064                 x = a[j - 1];
00065                 a[j - 1] = a[j];
00066                 a[j] = x;
00067                 k = j;
00068             }
00069         }
00070     }
00071     ub = k - 1;
00072     while (lb < ub);
00073 }
00074 } while (lb < ub);
00075 }
```

Используется в [main\(\)](#).

Граф вызова функции:

6.10 MySortings.h

См. документацию.

```

00001 #pragma once
00002
00009 template<class T>
00010 void bubbleSort(T a[], int size)
00011 {
00012     T x;
00013
00014     for (int i = 0; i < size; i++)
00015     {
00016         for (int j = size - 1; j > i; j--)
00017         {
00018             //меняем соседние элементы местами
00019             if (a[j - 1] > a[j])
00020             {
00021                 x = a[j - 1];
00022                 a[j - 1] = a[j];
00023                 a[j] = x;
00024             }
00025         }
00026     }
00027 }
```

```

00028
00035 template<class T>
00036 void shakerSort(T a[], int size)
00037 {
00038     int j;
00039     int k = size - 1;
00040     int lb = 1;
00041     int ub = size - 1;
00042     T x;
00043
00044     do
00045     {
00046         for (j = ub; j > 0; j--)
00047         {
00048             //проход справа налево
00049             if (a[j - 1] > a[j])
00050             {
00051                 x = a[j - 1];
00052                 a[j - 1] = a[j];
00053                 a[j] = x;
00054                 k = j;
00055             }
00056         }
00057
00058         lb = k + 1;
00059
00060         for (j = 1; j <= ub; j++)
00061         {
00062             //проход слева направо
00063             if (a[j - 1] > a[j])
00064             {
00065                 x = a[j - 1];
00066                 a[j - 1] = a[j];
00067                 a[j] = x;
00068                 k = j;
00069             }
00070         }
00071
00072         ub = k - 1;
00073
00074     } while (lb < ub);
00075 }
00076
00085 template<class T>
00086 void merge(T a[], int low, int mid, int high)
00087 {
00088     T* b = new T[high + 1 - low];
00089
00090     int h = low;
00091     int i = 0;
00092     int j = mid + 1;
00093
00094     while ((h <= mid) && (j <= high))
00095     {
00096         //выбираем меньший элемент
00097         if (a[h] <= a[j])
00098         {
00099             b[i] = a[h];
00100             h++;
00101         }
00102         else
00103         {
00104             b[i] = a[j];
00105             j++;
00106         }
00107
00108         i++;
00109     }
00110
00111     if (h > mid)
00112     {
00113         for (int k = j; k <= high; k++)
00114         {
00115             //копируем остаток правой части
00116             b[i] = a[k];
00117             i++;
00118         }
00119     }
00120     else
00121     {
00122         for (int k = h; k <= mid; k++)
00123         {
00124             //копируем остаток левой части
00125             b[i] = a[k];
00126             i++;
00127         }
00128     }

```

```

00129
00130     for (int k = 0; k <= high - low; k++)
00131     {
00132         //возвращаем элементы в исходный массив
00133         a[k + low] = b[k];
00134     }
00135     delete[] b;
00137 }
00138
00146 template<class T>
00147 void merge_sort(T a[], int low, int high)
00148 {
00149     if (low < high)
00150     {
00151         //делим массив на две части
00152         int mid = (low + high) / 2;
00153
00154         merge_sort(a, low, mid);
00155         merge_sort(a, mid + 1, high);
00156
00157         //сливаем отсортированные части
00158         merge(a, low, mid, high);
00159     }
00160 }

```

6.11 Файл Sorts/oper.cpp

#include "oper.h"

Граф включаемых заголовочных файлов для oper.cpp:

Функции

- bool operator< (Tovar a, Tovar b)
сравнивает два товара по возрастанию
- bool operator> (Tovar a, Tovar b)
сравнивает два товара по убыванию
- bool operator<= (Tovar a, Tovar b)
проверяет, что первый товар меньше или равен второму
- bool operator>= (Tovar a, Tovar b)
проверяет, что первый товар больше или равен второму

6.11.1 Функции

6.11.1.1 operator<()

```

bool operator< (
    Tovar a,
    Tovar b)

```

сравнивает два товара по возрастанию

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар меньше второго

См. определение в файле `oper.cpp` строка 3

```
00004 {  
00005     //сравниваем по названию  
00006     if (a.name != b.name)  
00007         return a.name < b.name;  
00008  
00009     //сравниваем по объему  
00010     if (a.volume != b.volume)  
00011         return a.volume < b.volume;  
00012  
00013     //сравниваем по стране  
00014     return a.country < b.country;  
00015 }
```

Перекрестные ссылки `Tovar::country`, `Tovar::name` и `Tovar::volume`.

6.11.1.2 operator<=()

```
bool operator<= (  
    Tovar a,  
    Tovar b)
```

проверяет, что первый товар меньше или равен второму

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар меньше или равен второму

См. определение в файле `oper.cpp` строка 22

```
00023 {  
00024     return !(a > b);  
00025 }
```

6.11.1.3 operator>()

```
bool operator> (  
    Tovar a,  
    Tovar b)
```

сравнивает два товара по убыванию

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар больше второго

См. определение в файле `oper.cpp` строка 17

```
00018 {  
00019     return b < a;  
00020 }
```


6.11.1.4 operator>=()

```
bool operator>= (
    Tovar a,
    Tovar b)
```

проверяет, что первый товар больше или равен второму

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар больше или равен второму

См. определение в файле [oper.cpp](#) строка 27

```
00028 {
00029     return !(a < b);
00030 }
```

6.12 oper.cpp

См. документацию.

```
00001 #include "oper.h"
00002
00003 bool operator < (Tovar a, Tovar b)
00004 {
00005     //сравниваем по названию
00006     if (a.name != b.name)
00007         return a.name < b.name;
00008
00009     //сравниваем по объему
00010     if (a.volume != b.volume)
00011         return a.volume < b.volume;
00012
00013     //сравниваем по стране
00014     return a.country < b.country;
00015 }
00016
00017 bool operator > (Tovar a, Tovar b)
00018 {
00019     return b < a;
00020 }
00021
00022 bool operator <= (Tovar a, Tovar b)
00023 {
00024     return !(a > b);
00025 }
00026
00027 bool operator >= (Tovar a, Tovar b)
00028 {
00029     return !(a < b);
00030 }
```

6.13 Файл Sorts/oper.h

```
#include <string>
```

Граф включаемых заголовочных файлов для oper.h: Граф файлов, в которые включается этот файл:

Классы

- `struct Tovar`
структура товара для экспорта

Функции

- `bool operator< (Tovar a, Tovar b)`
сравнивает два товара по возрастанию
- `bool operator> (Tovar a, Tovar b)`
сравнивает два товара по убыванию
- `bool operator<= (Tovar a, Tovar b)`
проверяет, что первый товар меньше или равен второму
- `bool operator>= (Tovar a, Tovar b)`
проверяет, что первый товар больше или равен второму

6.13.1 Функции

6.13.1.1 `operator<()`

```
bool operator< (
    Tovar a,
    Tovar b)
```

сравнивает два товара по возрастанию

Аргументы

a	первый товар
b	второй товар

Возвращает

`true`, если первый товар меньше второго

См. определение в файле `oper.cpp` строка 3

```
00004 {
00005     //сравниваем по названию
00006     if (a.name != b.name)
00007         return a.name < b.name;
00008
00009     //сравниваем по объему
00010     if (a.volume != b.volume)
00011         return a.volume < b.volume;
00012
00013     //сравниваем по стране
00014     return a.country < b.country;
00015 }
```

Перекрестные ссылки `Tovar::country`, `Tovar::name` и `Tovar::volume`.

6.13.1.2 operator<=()

```
bool operator<= (  
    Tovar a,  
    Tovar b)
```

проверяет, что первый товар меньше или равен второму

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар меньше или равен второму

См. определение в файле [oper.cpp](#) строка 22

```
00023 {  
00024     return !(a > b);  
00025 }
```

6.13.1.3 operator>()

```
bool operator> (  
    Tovar a,  
    Tovar b)
```

сравнивает два товара по убыванию

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар больше второго

См. определение в файле [oper.cpp](#) строка 17

```
00018 {  
00019     return b < a;  
00020 }
```

6.13.1.4 operator>=()

```
bool operator>= (
    Tovar a,
    Tovar b)
```

проверяет, что первый товар больше или равен второму

Аргументы

a	первый товар
b	второй товар

Возвращает

true, если первый товар больше или равен второму

См. определение в файле [oper.cpp](#) строка 27

```
00028 {
00029     return !(a < b);
00030 }
```

6.14 oper.h

См. документацию.

```
00001 #pragma once
00002 #include <string>
00003
00004 using namespace std;
00005
00009 struct Tovar
00010 {
00011     string name;
00012     string country;
00013     int volume;
00014     int money;
00015 };
00016
00023 bool operator < (Tovar a, Tovar b);
00024
00031 bool operator > (Tovar a, Tovar b);
00032
00039 bool operator <= (Tovar a, Tovar b);
00040
00047 bool operator >= (Tovar a, Tovar b);
```

6.15 Файл Sorts/shaker.cpp

Функции

- `template<class T>`
void [shakerSort](#) (T a[], long size)

6.15.1 Функции

6.15.1.1 shakerSort()

```
template<class T>
void shakerSort (
    T a[],
    long size)
```

См. определение в файле [shaker.cpp](#) строка 3

```
00004 {
00005     long j, k = size - 1;
00006     long lb = 1, ub = size - 1;
00007     T x;
00008
00009     do
00010     {
00011
00012         for (j = ub; j > 0; j--)
00013         {
00014             if (a[j - 1] > a[j])
00015             {
00016                 x = a[j - 1]; a[j - 1] = a[j]; a[j] = x;
00017                 k = j;
00018             }
00019         }
00020
00021         lb = k + 1;
00022
00023         for (j = 1; j <= ub; j++)
00024         {
00025             if (a[j - 1] > a[j])
00026             {
00027                 x = a[j - 1]; a[j - 1] = a[j]; a[j] = x;
00028                 k = j;
00029             }
00030         }
00031
00032         ub = k - 1;
00033     } while (lb < ub);
00034 }
```

6.16 shaker.cpp

См. документацию.

```
00001
00002
00003 template<class T> void shakerSort(T a[], long size)
00004 {
00005     long j, k = size - 1;
00006     long lb = 1, ub = size - 1;
00007     T x;
00008
00009     do
00010     {
00011
00012         for (j = ub; j > 0; j--)
00013         {
00014             if (a[j - 1] > a[j])
00015             {
00016                 x = a[j - 1]; a[j - 1] = a[j]; a[j] = x;
00017                 k = j;
00018             }
00019         }
00020
00021         lb = k + 1;
00022
00023         for (j = 1; j <= ub; j++)
00024         {
00025             if (a[j - 1] > a[j])
00026             {
00027                 x = a[j - 1]; a[j - 1] = a[j]; a[j] = x;
00028                 k = j;
00029             }
00030         }
00031
00032         ub = k - 1;
00033     } while (lb < ub);
00034 }
```


Предметный указатель

- bubble.cpp
 - bubbleSort, [13](#)
- bubbleSort
 - bubble.cpp, [13](#)
 - MySortings.h, [22](#)
- copyArray
 - main.cpp, [14](#)
- country
 - Tovar, [9](#)
- DataSetMaker/dsmaker.cpp, [11](#), [12](#)
- dsmaker.cpp
 - main, [11](#)
- main
 - dsmaker.cpp, [11](#)
 - main.cpp, [14](#)
- main.cpp
 - copyArray, [14](#)
 - main, [14](#)
 - readData, [16](#)
 - writeData, [16](#)
- merge
 - merge.cpp, [19](#)
 - MySortings.h, [22](#)
- merge.cpp
 - merge, [19](#)
 - merge_sort, [20](#)
- merge_sort
 - merge.cpp, [20](#)
 - MySortings.h, [23](#)
- money
 - Tovar, [9](#)
- MySortings.h
 - bubbleSort, [22](#)
 - merge, [22](#)
 - merge_sort, [23](#)
 - shakerSort, [24](#)
- name
 - Tovar, [10](#)
- oper.cpp
 - operator<, [27](#)
 - operator<=, [28](#)
 - operator>, [28](#)
 - operator>=, [28](#)
- oper.h
 - operator<, [30](#)
 - operator<=, [30](#)
 - operator>, [31](#)
 - operator>=, [31](#)
- operator<
 - oper.cpp, [27](#)
 - oper.h, [30](#)
- operator<=
 - oper.cpp, [28](#)
 - oper.h, [30](#)
- operator>
 - oper.cpp, [28](#)
 - oper.h, [31](#)
- operator>=
 - oper.cpp, [28](#)
 - oper.h, [31](#)
- readData
 - main.cpp, [16](#)
- shaker.cpp
 - shakerSort, [33](#)
- shakerSort
 - MySortings.h, [24](#)
 - shaker.cpp, [33](#)
- Sorts/bubble.cpp, [12](#), [13](#)
- Sorts/main.cpp, [13](#), [17](#)
- Sorts/merge.cpp, [19](#), [21](#)
- Sorts/MySortings.h, [21](#), [25](#)
- Sorts/oper.cpp, [27](#), [29](#)
- Sorts/oper.h, [29](#), [32](#)
- Sorts/shaker.cpp, [32](#), [33](#)
- Tovar, [9](#)
 - country, [9](#)
 - money, [9](#)
 - name, [10](#)
 - volume, [10](#)
- volume
 - Tovar, [10](#)
- writeData
 - main.cpp, [16](#)
- Содержание директории DataSetMaker, [7](#)
- Содержание директории Sorts, [7](#)